⚽ MCP Servers — Football Prediction Platform

Guia de Servidores MCP para a Arquitetura de Previsão de Futebol

Versão: 1.0 — Março 2026
Referência: Model Context Protocol — Documentação Oficial
Registro MCP: github.com/modelcontextprotocol/servers

📌 Contexto

O Model Context Protocol (MCP) é um protocolo aberto criado pela Anthropic em novembro de 2024 e posteriormente doado à Linux Foundation (Agentic AI Foundation — AAIF). Ele padroniza a integração entre LLMs e fontes de dados/ferramentas externas através de uma interface universal baseada em JSON-RPC 2.0.

Este documento mapeia os **melhores MCP Servers disponíveis** para cada componente da arquitetura do Football Prediction Platform, organizados por camada funcional.

🗺️ Mapa de Cobertura: Componente → MCP Server

Componente da Arquitetura	MCP Server Recomendado	Tipo
Apache Airflow	`astro-airflow-mcp` (Astronomer)	Oficial / Community
PostgreSQL	`postgres-mcp` (Crystal DBA)	Community
dbt Core	`dbt-mcp` (dbt Labs)	Oficial
MLflow	`mlflow-mcp` (kkruglik)	Community
Docker	`mcp-server-docker` (ckreiling)	Community
Apache Superset	`superset-mcp` (Ponder)	Community
Git (Versionamento)	`mcp-server-git` (Referência MCP)	Referência Oficial
Filesystem	`mcp-server-filesystem` (Referência MCP)	Referência Oficial
Fetch (Web/APIs)	`mcp-server-fetch` (Referência MCP)	Referência Oficial
Sequential Thinking	`mcp-server-sequential-thinking` (Referência MCP)	Referência Oficial

🔹 1. Apache Airflow MCP — Orquestração

Servidor Recomendado: `astro-airflow-mcp`

Repositório: github.com/astronomer/astro-airflow-mcp
Mantido por: Astronomer (empresa por trás do Astro CLI)
Compatibilidade: Airflow 2.x e 3.x (detecção automática de versão)

Por que este e não outros? Existem ao menos 6 MCP servers para Airflow no ecossistema. O `astro-airflow-mcp` se destaca por ser mantido pela Astronomer (principal contribuidora do ecossistema Airflow), suportar ambas as versões do Airflow com adapter pattern, e oferecer ferramentas consolidadas como `explore_dag`, `diagnose_dag_run` e `get_system_health` que reduzem o número de chamadas necessárias.

Tools Disponíveis:

- **DAG Management:** list, get details, get source code, stats, warnings, import errors, trigger, pause/unpause
- **Task Management:** list, get details, get task instances, get logs
- **Pool Management:** list, get details
- **Variable Management:** list, get specific variables
- **Connection Management:** list connections (credenciais excluídas)
- **Asset/Dataset Management:** data lineage unificado
- **Ferramentas Consolidadas:**
  - `explore_dag` — informações completas de uma DAG em uma única chamada
  - `diagnose_dag_run` — debug de DAG runs com falha, incluindo detalhes de task instances
  - `get_system_health` — overview do sistema com health, errors e warnings

Configuração:

```json
{
  "mcpServers": {
    "airflow": {
      "command": "uvx",
      "args": ["astro-airflow-mcp", "--transport", "stdio"],
      "env": {
        "AIRFLOW_API_URL": "http://localhost:8080",
        "AIRFLOW_USERNAME": "admin",
        "AIRFLOW_PASSWORD": "admin"
      }
    }
  }
}
```

Aplicação no Projeto: Orquestração da DAG principal `ingest_data → spark_processing → dbt_run → dbt_test → train_model → predictions`. O agente pode monitorar, diagnosticar falhas e re-triggerar DAGs via linguagem natural.

Alternativa: `mcp-server-apache-airflow`

Repositório: github.com/yangkyeongmo/mcp-server-apache-airflow

Opção mais leve, com suporte a modo read-only e seleção granular de APIs (dag, dagrun, taskinstance, variable, xcom, etc.). Útil para ambientes onde se deseja restringir o escopo de acesso.

🔹 2. PostgreSQL MCP — Serving Layer

Servidor Recomendado: `postgres-mcp` **(Postgres MCP Pro)**

Repositório: github.com/crystaldba/postgres-mcp
Mantido por: Crystal DBA
Diferencial: Vai além da execução de queries — inclui ferramentas de análise de performance e tuning automático de índices.

Por que este? Enquanto existem dezenas de MCP servers para PostgreSQL (incluindo o servidor de referência oficial), o Postgres MCP Pro adiciona capacidades que são críticas para um projeto de dados em produção: explain plans com índices hipotéticos, health checks, e um algoritmo de tuning automático de índices baseado no Anytime Algorithm do SQL Server Database Tuning Advisor.

Tools Disponíveis:

- `list_schemas` — lista schemas disponíveis
- `list_objects` — lista tabelas, views, sequences e extensions dentro de um schema
- `get_object_details` — detalhes de tabelas (colunas, constraints, índices)
- `execute_sql` — execução de SQL (com restrição read-only em modo restrito)
- `explain_query` — plano de execução com modelo de custo do planner PostgreSQL, suporte a índices hipotéticos

Configuração:

```json
{
  "mcpServers": {
    "postgres": {
      "command": "uvx",
      "args": ["postgres-mcp"],
      "env": {
        "DATABASE_URL": "postgresql://user:password@localhost:5432/football_predicti
      }
    }
  }
}
```

Aplicação no Projeto: Consultas nas tabelas `matches`, `teams`, `predictions`, `team_strength`. Análise de performance de queries usadas pelo Superset e pela API FastAPI. Tuning de índices para as tabelas mais consultadas.

Alternativa: `pg-mcp-server`

Repositório: github.com/stuzero/pg-mcp-server

Implementação completa com SSE transport, suporte multi-database simultâneo, e contexto de extensões PostgreSQL (PostGIS, pgvector). Útil se o projeto evoluir para dados geoespaciais (localização de estádios, etc.).

🔹 3. dbt Core MCP — Modelagem e Transformação

Servidor Recomendado: `dbt-mcp` **(Oficial dbt Labs)**

Repositório: github.com/dbt-labs/dbt-mcp
Mantido por: dbt Labs (criadores do dbt)
Status: Ativamente mantido, releases frequentes

Por que este? Este é o servidor MCP oficial mantido pela dbt Labs. Oferece integração com dbt Core, dbt Fusion e dbt Platform. Inclui ferramentas de geração de código, lineage, documentação e execução.

Tools Disponíveis:

- **Execução e Compilação:**
  - `build` — executa models, tests, snapshots e seeds em ordem do DAG
  - `compile` — gera SQL executável a partir de models/tests/analyses
  - `run` — executa models para materialização no banco
  - `show` — executa SQL e retorna resultados
  - `test` — executa testes do projeto
  - `parse` — valida arquivos do projeto
- **Discovery e Lineage:**
  - `get_lineage_dev` — lineage do manifest.json local com filtros de tipo e profundidade
  - `get_node_details_dev` — detalhes de nós (models, seeds, snapshots, sources)
  - `list` — lista recursos do projeto por tipo
- **Geração de Código:**
  - `generate_model_yaml` — gera YAML de modelo com colunas e descrições herdadas
  - `generate_source` — gera YAML de source por introspecção do banco
  - `generate_staging_model` — gera SQL de staging model a partir de tabela source
- **Documentação:**
  - `docs` — gera documentação do projeto
  - `search_product_docs` — busca na documentação oficial do dbt
  - `get_product_doc_pages` — obtém conteúdo completo de páginas da documentação

Configuração:

```json
{
  "mcpServers": {
    "dbt": {
      "command": "uvx",
      "args": ["dbt-mcp"],
      "env": {
        "DBT_PROJECT_DIR": "/path/to/football-prediction-mcp/dbt",
        "DBT_PROFILES_DIR": "~/.dbt"
      }
    }
  }
}
```

Aplicação no Projeto: Gestão das camadas `staging/`, `intermediate/` e `marts/`. Geração e manutenção do modelo principal `fct_match_features`. Execução de testes de qualidade de dados. Exploração do lineage entre as fontes de dados e as features de ML.

🔹 4. MLflow MCP — Machine Learning Lifecycle

Servidor Recomendado: `mlflow-mcp` **(kkruglik)**

Repositório: github.com/kkruglik/mlflow-mcp
Stars/Atividade: Projeto community bem documentado

Tools Disponíveis:

- **Experiment Management:** list e search de experiments, descoberta de métricas e parâmetros disponíveis
- **Run Analysis:** detalhes de runs, queries com filtros, identificação de melhores modelos
- **Metrics & Parameters:** histórico de métricas, comparação de parâmetros entre runs
- **Model Registry:** CRUD operations nos modelos registrados

Configuração:

```json
{
  "mcpServers": {
    "mlflow": {
      "command": "uvx",
      "args": ["mlflow-mcp"],
      "env": {
        "MLFLOW_TRACKING_URI": "http://localhost:5000"
      }
    }
  }
}
```

Aplicação no Projeto: Gestão do ciclo de vida dos modelos Poisson, Gradient Boosting e Modelo Híbrido. Comparação de métricas (Log Loss, Brier Score, ROI simulado) entre versões. Registro e promoção de modelos para produção. Tracking de experiments de feature engineering.

Alternativa: `mlflow-mcp-server` **(yesid-lopez)**

Repositório: github.com/yesid-lopez/mlflow-mcp-server

Estrutura modular com tools separados por domínio (experiment_tools, run_tools, registered_models). Oferece funcionalidade similar com organização de código ligeiramente diferente.

🔹 5. Docker MCP — Infraestrutura

Servidor Recomendado: `mcp-server-docker` **(ckreiling)**

Repositório: github.com/ckreiling/mcp-server-docker
Transporte: stdio (local) e SSE (remoto)

Tools Disponíveis:

- Gestão de containers (criar, listar, start, stop, restart, remove)
- Gestão de imagens (pull, list, inspect)
- Compose stacks (deploy, manage)
- Container logs
- Execução de comandos dentro de containers

Configuração:

```json
{
  "mcpServers": {
    "docker": {
      "command": "uvx",
      "args": ["mcp-server-docker"],
      "env": {
        "DOCKER_HOST": "unix:///var/run/docker.sock"
      }
    }
  }
}
```

Aplicação no Projeto: Gestão dos serviços `postgres`, `airflow`, `spark`, `superset`, `mlflow` via `docker-compose`. Monitoramento de health dos containers. Troubleshooting de serviços que falharam. Deploy e rebuild de serviços específicos.

Alternativa: Docker MCP Gateway (Oficial Docker)

Repositório: github.com/docker/mcp-gateway

O Docker MCP Gateway é o projeto oficial da Docker Inc. que funciona como um gateway unificado para múltiplos MCP servers. Cada servidor MCP local roda em um container Docker isolado. Requer Docker Desktop 4.59+ e oferece secrets management integrado. É a opção mais robusta para ambientes de produção.

🔹 6. Apache Superset MCP — Visualização / BI

Servidor Recomendado: `superset-mcp` **(Ponder)**

Repositório: github.com/ponderedw/superset-mcp
Mantido por: Ponder (também mantém o Airflow MCP Ponder)

Tools Disponíveis:

- Listar e consultar dashboards
- Obter detalhes de dashboards
- Criar e atualizar dashboards
- Consultar metadados de charts e datasets
- Exploração de metadata em linguagem natural

Configuração:

```json
{
  "mcpServers": {
    "superset": {
      "command": "docker",
      "args": [
        "run", "-i", "--rm",
        "-e", "SUPERSET_API_URL",
        "-e", "SUPERSET_USERNAME",
        "-e", "SUPERSET_PASSWORD",
        "pondered/superset-mcp:latest"
      ],
      "env": {
        "SUPERSET_API_URL": "http://host.docker.internal:8088",
        "SUPERSET_USERNAME": "superset_admin",
        "SUPERSET_PASSWORD": "superset"
      }
    }
  }
}
```

Aplicação no Projeto: Gestão dos dashboards de Probabilidades (Win/Draw/Loss), Ranking de times, Value bets e ROI histórico. Refresh automatizado após execução do pipeline. Exploração de metadados dos charts via linguagem natural.

Alternativa: `superset-mcp-plugins` **(Ponder — versão com AI Assistant)**

Repositório: github.com/ponderedw/superset-mcp-plugins

Versão estendida que adiciona um AI Assistant dentro do Superset com suporte a LangChain/LangGraph, dbt graph via Neo4j/FalkorDB, e múltiplos providers de LLM (OpenAI, Anthropic, AWS Bedrock).

🔹 7. Servidores MCP de Referência (Utilitários)

Estes são os servidores mantidos pelo MCP Steering Group como implementações de referência.

7.1 Git MCP

Repositório: github.com/modelcontextprotocol/servers (pasta `src/git`)

Tools para leitura, busca e manipulação de repositórios Git. Útil para versionamento do projeto, tracking de mudanças em DAGs, modelos dbt e scripts Spark.

```json
{
  "mcpServers": {
    "git": {
      "command": "uvx",
      "args": ["mcp-server-git", "--repository", "/path/to/football-prediction-mcp"]
    }
  }
}
```

7.2 Filesystem MCP

Operações seguras de arquivo com controle de acesso configurável. Essencial para manipulação de arquivos de dados, configurações e artefatos do pipeline.

```json
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y", "@modelcontextprotocol/server-filesystem",
        "/path/to/football-prediction-mcp"
      ]
    }
  }
}
```

7.3 Fetch MCP

Busca e conversão de conteúdo web para uso eficiente por LLMs. Útil para integração com APIs de futebol e odds.

```json
{
  "mcpServers": {
    "fetch": {
      "command": "uvx",
      "args": ["mcp-server-fetch"]
    }
  }
}
```

7.4 Sequential Thinking MCP

Raciocínio dinâmico e reflexivo para resolução de problemas complexos. Útil para debugging de pipelines e análise de resultados de modelos.

```json
{
  "mcpServers": {
    "sequential-thinking": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-sequential-thinking"]
    }
  }
}
```

🧠 Configuração Consolidada — MCP Orchestrator

Abaixo, a configuração unificada de todos os MCPs para o Claude Desktop ou qualquer cliente MCP compatível:

```json
{
  "mcpServers": {
    "airflow": {
      "command": "uvx",
      "args": ["astro-airflow-mcp", "--transport", "stdio"],
      "env": {
        "AIRFLOW_API_URL": "http://localhost:8080",
        "AIRFLOW_USERNAME": "admin",
        "AIRFLOW_PASSWORD": "admin"
      }
    },
    "postgres": {
      "command": "uvx",
      "args": ["postgres-mcp"],
      "env": {
        "DATABASE_URL": "postgresql://user:password@localhost:5432/football_predicti
      }
    },
    "dbt": {
      "command": "uvx",
      "args": ["dbt-mcp"],
      "env": {
        "DBT_PROJECT_DIR": "./dbt",
        "DBT_PROFILES_DIR": "~/.dbt"
      }
    },
    "mlflow": {
      "command": "uvx",
      "args": ["mlflow-mcp"],
      "env": {
        "MLFLOW_TRACKING_URI": "http://localhost:5000"
      }
    },
    "docker": {
      "command": "uvx",
      "args": ["mcp-server-docker"]
    },
    "superset": {
      "command": "docker",
      "args": [
        "run", "-i", "--rm",
        "-e", "SUPERSET_API_URL",
        "-e", "SUPERSET_USERNAME",
        "-e", "SUPERSET_PASSWORD",
        "pondered/superset-mcp:latest"
      ],
      "env": {
```

```
        "SUPERSET_API_URL": "http://host.docker.internal:8088",
        "SUPERSET_USERNAME": "superset_admin",
        "SUPERSET_PASSWORD": "superset"
      }
    },
    "git": {
      "command": "uvx",
      "args": ["mcp-server-git", "--repository", "."]
    },
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "."]
    },
    "fetch": {
      "command": "uvx",
      "args": ["mcp-server-fetch"]
    },
    "sequential-thinking": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-sequential-thinking"]
    }
  }
}
```

🔄 Fluxo de Orquestração Inteligente via MCP

Exemplo: "Atualize as previsões de hoje"

```
Agente (LLM)
  |
  ├─ 1. Docker MCP → Verifica health dos containers
  |
  ├─ 2. Airflow MCP → Trigger DAG "ingest_data"
  |     └─ Monitora status até conclusão
  |
  ├─ 3. Airflow MCP → Verifica conclusão do Spark job (via task instance)
  |
  ├─ 4. dbt MCP → Executa `dbt run` (materializa fct_match_features)
  |     └─ dbt MCP → Executa `dbt test` (validação)
  |
  ├─ 5. MLflow MCP → Carrega modelo "champion" do registry
  |     └─ PostgreSQL MCP → Insere previsões na tabela predictions
  |
  ├─ 6. Superset MCP → Refresh dos dashboards
  |
  └─ 7. PostgreSQL MCP → Query de validação final
```

⚠️ Considerações de Segurança

O ecossistema MCP ainda está em amadurecimento no aspecto de segurança. Pesquisas de 2025 identificaram que grande parte dos MCP servers expostos à internet não possuíam autenticação. Recomendações para o projeto:

1. **Execução local:** Rodar todos os MCP servers localmente via Docker socket, evitando exposição à rede.
2. **Modo read-only:** Para MCPs de database e Airflow, utilizar modo read-only em ambientes de produção.
3. **Docker MCP Gateway:** Considerar o uso do Docker MCP Gateway oficial para isolamento de containers e gestão centralizada de secrets.
4. **Credenciais:** Nunca hardcodar credenciais nos JSONs de configuração. Utilizar variáveis de ambiente ou Docker secrets.
5. **Spec atualizada:** A especificação MCP de novembro de 2025 introduziu melhorias de autenticação com OAuth Resource Servers e Resource Indicators (RFC 8707).

📚 Recursos de Aprendizado

Recurso	Link
Especificação MCP (2025-11-25)	modelcontextprotocol.io/specification
Repositório oficial de servers	github.com/modelcontextprotocol/servers
Curso Anthropic — Intro to MCP (Python SDK)	anthropic.skilljar.com
MCP Registry (browse público)	github.com/modelcontextprotocol/servers#community-servers
Docker MCP Registry (oficial Docker)	github.com/docker/mcp-registry
Blog MCP — 1 ano do protocolo	blog.modelcontextprotocol.io
SDKs disponíveis	Python, TypeScript, C#, Java, Kotlin, Ruby

🚀 Próximos Passos

1. **MCP Server customizado para APIs de Futebol:** Construir um MCP server com FastMCP (Python) para expor as APIs de futebol (API-Football, Odds API) como tools padronizados.
2. **MCP Server para Spark:** Ainda não existe um MCP server community maduro para Apache Spark. A recomendação é construir um custom server usando o Python SDK que faça submit de jobs via `spark-submit` ou REST API do Spark.
3. **Kafka MCP (futuro):** Para a evolução com streaming, monitorar o ecossistema para um MCP server de Kafka (alguns projetos early-stage já existem).
4. **Feature Store MCP:** Quando implementar o Feature Store, avaliar integração via MCP para exposição das features ao agente.

Documento gerado em: Março 2026
Fontes: Documentação oficial MCP, repositórios GitHub dos respectivos projetos, MCP Registry público.